

Agente Multimodal

de Análisis de Documentos Académicos

Memoria técnica de la práctica final

Universidad Europea de Madrid

Grado en Ingeniería de la Ciberseguridad

Inteligencia Artificial · Curso académico 2025-2026

Iris Carolina Fernández González

Mayo de 2026

Índice

Introducción	3
Arquitectura del sistema	3
Decisiones técnicas	3
Prompt engineering	4
Ejemplos de uso	4
Limitaciones y mejoras futuras	6

Introducción

Para esta práctica he desarrollado un agente que analiza documentos académicos. La idea salió de algo que me pasa a menudo: cuando tengo que estudiar varios artículos científicos pierdo bastante tiempo en localizar lo importante de cada uno. Por eso quise montar una herramienta que, a partir de un artículo en PDF o texto, me devuelva un resumen claro, me saque los datos principales, me compare dos trabajos y me deje hacerle preguntas sobre el contenido.

También me decidí por este enfoque por la privacidad: como todo se ejecuta en mi propio equipo, los documentos no salen de él. Estudiando ciberseguridad, ese detalle me parecía importante. He preparado tres escenarios de uso, que enseño más adelante.

Arquitectura del sistema

El agente sigue un flujo bastante sencillo. Primero leo el documento y lo paso a texto; después construyo un prompt que junta ese contenido con lo que pregunta el usuario; ese prompt se envía al modelo que tengo corriendo en local con Ollama, y la respuesta vuelve ya ordenada en un formato estructurado. En la parte de preguntas guardo el historial de la conversación para poder hacer preguntas de seguimiento sin repetir el contexto.



Pipeline del agente.

He repartido el código en varios archivos para que cada uno tenga una tarea clara:

- ingest.py lee el documento (PDF con PyMuPDF, CSV con pandas, texto e imagen).
- prompts.py guarda un prompt distinto para cada funcionalidad.
- agent.py orquesta el modelo con LangChain y devuelve la salida estructurada.
- config.py reúne la configuración (modelo, servidor y límites).
- app.py es la interfaz web y cli.py la versión por línea de comandos.

Decisiones técnicas

El modelo que he elegido

He usado Llama 3.1 8B con Ollama. Como mi agente trabaja sobre todo con texto, no necesitaba un modelo multimodal pesado y preferí uno de texto, que va más fino en esta tarea. Lo elegí también por una cuestión práctica: cuantizado ocupa unos 4,9 GB y me cabe sin problemas en la GPU de mi portátil (una RTX 4060 de 8 GB); un modelo de visión de 11B se me quedaría muy justo. Aun así, dejé preparada la opción de activar un modelo de visión aparte para las imágenes.

Por qué Ollama y no LM Studio

Miré las dos opciones, pero me quedé con Ollama porque se maneja desde la terminal y levanta

una API compatible con la de OpenAI, lo que me facilitó mucho conectarlo con LangChain y automatizar las pruebas. LM Studio tiene una interfaz más cómoda, pero para llamar al modelo desde el código Ollama me resultó más directo.

Librerías que he usado

- LangChain, para la orquestación (chains, agente, herramientas y prompts).
- Pydantic, para la salida estructurada.
- PyMuPDF, para extraer el texto de los PDF.
- pandas, para leer los CSV.
- Streamlit, para la interfaz.

Prompt engineering

En vez de usar un único prompt para todo, he escrito uno distinto para cada tarea. Todos parten del mismo rol, que fija el idioma y, sobre todo, le pide al modelo que no se invente nada que no esté en el documento; ese era uno de los fallos que más quería evitar.

- Resumen: saca el resumen, los objetivos, la metodología, los hallazgos y las limitaciones.
- Metadatos: saca el título, los autores, el año, la publicación, las palabras clave, los datasets y las métricas.
- Comparación: extrae una ficha de cada documento y redacta una conclusión.
- Preguntas: responde usando el documento y el historial de la conversación.

Por ejemplo, el prompt de resumen junta ese rol con la tarea concreta:

```
Eres un asistente experto en analizar artículos científicos. Respondes siempre en español. Te basas solo en el contenido del documento; si algo no aparece, indica 'No consta en el documento' en lugar de inventarlo. Resume el artículo: resumen, objetivos, metodología, hallazgos y limitaciones.
```

Cada prompt lo conecto con la salida estructurada de LangChain, así me aseguro de que el modelo devuelva justo los campos que espero (el informe, los metadatos o la comparación).

Ejemplos de uso

Para probar la herramienta cogí dos artículos científicos reales sobre detección de intrusiones en red con aprendizaje automático. Estas son tres capturas con la entrada, el procesamiento y el resultado de cada escenario.

Agente Multimodal

Resumen Metadatos Comparativa Preguntas

RESUMEN

El artículo describe la importancia de mantener actualizado un Sistema de Detección de Intrusiones (IDS) para proteger la confidencialidad, integridad y disponibilidad de los servicios en una red. El autor utiliza el conjunto de datos KDD para evaluar diferentes técnicas de aprendizaje automático, específicamente J48, MLP y Redes Bayesianas, para detectar y clasificar ataques como DOS, R2L, U2R y PROBE. Los resultados muestran que el clasificador J48 alcanza la mayor tasa de precisión en la detección de estos ataques.

OBJETIVOS

- Evaluar diferentes técnicas de aprendizaje automático para detectar ataques en redes
- Comparar el rendimiento de varios clasificadores en la detección de ataques (DOS, R2L, U2R, y PROBE) en el conjunto de datos KDD

METODOLOGÍA

El estudio utiliza el conjunto de datos KDD, que incluye diferentes tipos de ataques. Se realizó un proceso de preprocesamiento para preparar el conjunto de datos para la clasificación. Luego se evaluaron tres clasificadores: J48, MLP y Bayes Network. El rendimiento de cada clasificador se evaluó mediante métricas como precisión y tasa de detección.

HALLAZGOS

- El clasificador J48 logró el mayor índice de precisión en la detección de ataques (DOS, R2L, U2R, y PROBE) en el conjunto de datos KDD
- El clasificador MLP también mostró un buen rendimiento en la detección de ataques.
- El estudio encontró que el clasificador J48 es más preciso que los otros dos clasificadores evaluados.

LIMITACIONES

- No se menciona si se realizaron pruebas de validación cruzada para evaluar el rendimiento de los clasificadores.
- No se discute la posible influencia de la selección del conjunto de datos en los resultados obtenidos.

Resumen estructurado de un artículo.

Agente Multimodal

Resumen Metadatos Comparativa Preguntas

Aspecto	Documento A	Documento B
Objetivo	Evaluar la eficacia de diferentes técnicas de aprendizaje automático para detectar ataques de intrusión en redes	Evaluar la generalizabilidad de conjuntos de características estándar para sistemas de detección de intrusiones basados en aprendizaje automático
Método	Utilizar el conjunto de datos KDD preprocesado con tres clasificadores (J48, MLP y Bayes Network) para evaluar su capacidad para detectar ataques de DOS, R2L, U2R y PROBE	Comparar dos conjuntos de características (NetFlow y CICFlowMeter) en tres datasets diferentes utilizando técnicas de aprendizaje automático y explicación de modelos
Datos	Conjunto de datos KDD, que incluye 148.758 instancias de diferentes tipos de ataques	Tres datasets de detección de intrusos: CSE-CIC-IDS2018, BoT-IoT y ToN-IoT, así como dos nuevos datasets generados en formato CICFlowMeter
Resultados	El clasificador J48 logró la mayor tasa de precisión para detectar y clasificar todos los ataques del conjunto de datos KDD	La investigación encontró que el conjunto de características NetFlow supera al conjunto de características CICFlowMeter en términos de precisión de detección en varios escenarios de ataque.

CONCLUSIÓN

La diferencia clave entre los dos trabajos es que el Documento A se centra en evaluar la eficacia de diferentes técnicas de aprendizaje automático para detectar ataques de intrusión en redes, utilizando un conjunto de datos específico (KDD), mientras que el Documento B se enfoca en evaluar la generalizabilidad de conjuntos de características estándar para sistemas de detección de intrusiones basados en aprendizaje automático. El Documento A es mejor para su objetivo porque proporciona una evaluación exhaustiva de diferentes técnicas de aprendizaje automático y un conjunto de datos amplio (148.758 instancias), lo que permite obtener resultados precisos y generalizables. El clasificador J48 logró la mayor tasa de precisión en este estudio. Por otro lado, el Documento B es mejor para su objetivo porque proporciona una evaluación comparativa de dos conjuntos de características estándar (NetFlow y CICFlowMeter) en diferentes escenarios de ataque, utilizando tres datasets diferentes. Esto permite evaluar la generalizabilidad de los resultados y obtener conclusiones más robustas. En resumen, el Documento A es mejor para evaluar la eficacia de técnicas de aprendizaje automático, mientras que el Documento B es mejor para evaluar la generalizabilidad de conjuntos de características estándar.

Comparativa de dos artículos.

Agente Multimodal

ResumenMetadatosComparativaPreguntas

¿Qué conjunto de datos utiliza el artículo?

El artículo utiliza el conjunto de datos KDD (Knowledge Discovery in Databases).

¿Y cuáles son sus limitaciones?

No consta en el documento que se mencionen las limitaciones del conjunto de datos KDD. Sin embargo, se menciona que el modelo propuesto por otros autores tiene una falta de precisión en la detección de ataques poco frecuentes.

Preguntas de seguimiento manteniendo el contexto.

Limitaciones y mejoras futuras

Estoy contenta con el resultado, pero soy consciente de que hay cosas que se pueden mejorar. Lo que mejor funciona, en mi experiencia, es el resumen y la extracción de datos: el agente es bastante fiel al texto y la memoria de la conversación responde bien a las preguntas de seguimiento.

Donde le veo más límites es en los documentos muy largos, porque recorto el texto para no pasarme de la ventana de contexto del modelo y se pierde parte del final. También noto que el modelo local, aunque cumple, se queda algo por detrás de uno comercial cuando la extracción es muy fina. Y la comparación, al usar un modelo pequeño, alguna vez dejaba campos sin rellenar; lo resolví haciendo que reintente, aunque no es una solución perfecta.

Si tuviera más tiempo, lo primero que haría sería partir los documentos largos y recuperar solo las partes relevantes, que es la idea del RAG. También me gustaría que cada respuesta enlazara con el fragmento del que sale, para poder fiarme más de ella, y poder exportar el informe desde la propia interfaz.